TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE

Part 5

Simulation and controls: SimuGraph, VHIDs, and where your Lua goes

In TSC the sim blueprint and the Lua script gave the locomotive its character. In TSW that work splits into the **SimuGraph** (a node graph of axles, motors, cylinders and valves integrated every frame), **VHID** control pairs, and a thin Blueprint event graph that copies control values into the graph. This part builds the minimal drivable set and shows how a TSC script translates.

In this part

1. The Simulation Asset and its node types
2. The minimal SimuGraph that drives
3. Parameter links and presets
4. VHID: a logic half and a click half
5. The three controls a locomotive needs
6. The RVM event graph: glue, not simulation
7. What happens to your TSC Lua
8. Simple Physics: the fallback

Node types, parameter names and Thomas's graph were read live from `SA_Thomas` (36 nodes, RK4) and the VHID fields from his components (**verified**). Whether the SimuGraph editor's palette offers every node type to new assets, and the exact Blueprint node titles for reading and writing parameters, were not opened during the inspection (**check**).

1 · The Simulation Asset and its node types

Create it with right-click ▸ Miscellaneous ▸ **Simulation Asset** in `Simulation/` and assign it to the RVM's Simulation component. Double-clicking opens the SimuGraph editor. The asset holds `PresetConfigData[]` (named configurations with Parked / RunningStopped / RunningActive snapshots) and the runtime graph: a `Nodes[]` array, each with a name, a struct type such as `SimulationNode_Axle`, and pins connected to other nodes. **verified**

Node type	Does	Parameters seen
`Axle`	A wheelset. Takes torque from the gearbox and brake force from pads, reports speed. The physics component's `ConnectedWheels` points here. Settings on the node: X offset from its parent (bogie or body), track **gauge** (1435 or 1520 mm), **wheel radius**, axle-to-track gap, suspension. **community**	SpeedAtRail_MS, WheelRotation_DEG, VelocityAtGround_MS
`Bogie`	A bogie pivot: X offset of the pivot from the vehicle centre; its axles name it as their parent. The ABP's Bogies entry points at it. **community**	
`ElectricalMotorV2`	Voltage/current to torque; polarity input for direction.	ReverseFieldPolarity
`ChopperV2`	Power controller between a voltage source and the motor; its position is the throttle.	CurrentPosition
`VoltageSource`, `Gearbox`	Ideal supply; ratio motor→axle.	
`BrakeRigging`, `BrakePad`, `BrakeCylinder`	Cylinder force → rigging → friction on one axle.	
`DirectBrakeControl`	Driver's brake valve model.	BrakeInputSignal, BrakeHoldActive
`AirCompressor`, `AirReservoir`, `AirPipe`, `AirValve`, `AirHose`, `AirExhaust`, `AirOneWayValve`, `LimitingValve`	The pneumatic toolkit. Hoses connect to the next vehicle's graph when coupled.	
`ValueConverter`	A plain input slot: Blueprints write `InputValue`; it exposes `OutValue0..n`, optionally remapped. **This is how controls enter the graph.**	InputValue, OutValue0, OutValue1
`ValueTransition`, `Expression`, `RateOfChange`	Smooth toward a target; free-form maths; derivative (HUD accelerometer).	TargetPosition, Result, InputValue

Dovetail's other locomotives show many more (diesel engine, traction motor sets, generators, exhausters, EP brake controllers, TPWS/AWS blocks); the ones above are the ones guaranteed to exist in your editor build because Thomas uses them.

2 · The minimal SimuGraph that drives

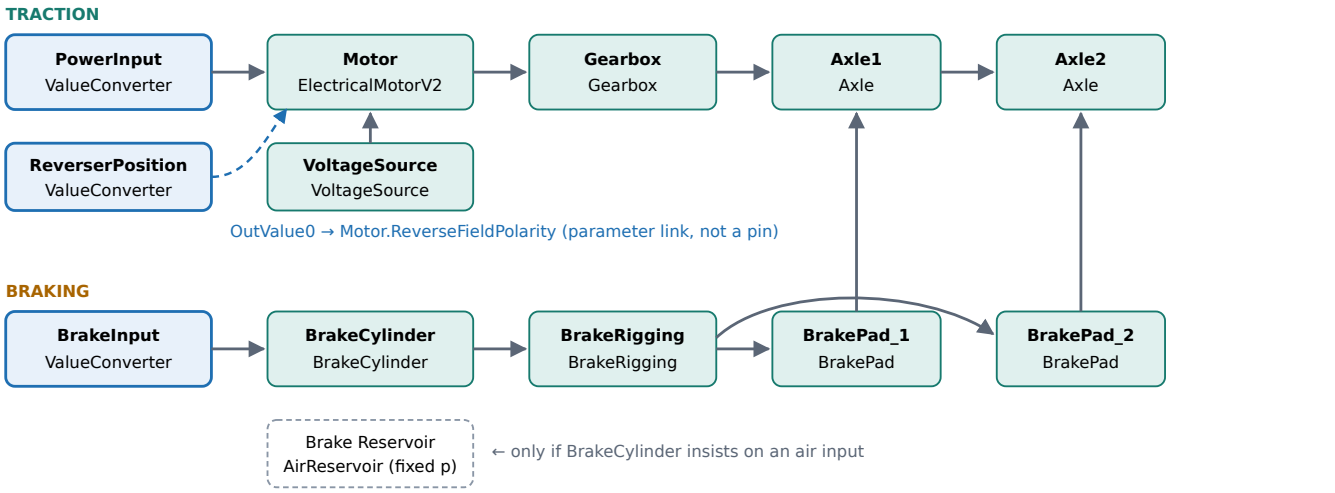


Figure 5.1 · Throttle, reverser, brake, two axles: still drivable. Thomas's real graph has 36 nodes (a chopper, a power ramp, a direct brake valve with apply/release valves, a compressor, main reservoir, pipes and hoses to the next vehicle); this is its skeleton.

1. `Axle1`, `Axle2`: set wheel radius (build sheet `axles[].radius_m`) and mass per axle; chain their axle pins. The physics component's `ConnectedWheels` is filled at runtime from these.
2. `VoltageSource → Motor → Gearbox → Axle1`.
3. `PowerInput`: map InputValue 0..1 to the motor's control pin (on Thomas this goes through a Chopper; try the motor's power input first, insert a `ChopperV2` if the node refuses).
4. `ReverserPosition.OutValue0 → Motor.ReverseFieldPolarity` via the parameter-link table (§3).
5. `BrakeInput → BrakeCylinder → BrakeRigging → BrakePad_1/2 → Axle1/2`.
6. Save; record a Default configuration's **RunningStopped** preset with brake applied and throttle at zero.

Steam, diesel, electric

Thomas is labelled “Steam” for the player but is simulated as an electric motor and a chopper: the graph only has to *feel* right. Start every loco this way; swap in the diesel-engine or boiler blocks from the node palette when the rest works. **verified**

A hauled wagon's graph is smaller still

The streamer's hopper rolled behind a locomotive with only six nodes: two `Bogie` nodes with their pivot offsets (the first at X = −5.16 m, the second positive) and four `Axle` nodes, each with gauge 1520 mm, the wheel radius, an offset of about ±0.95 m from its bogie, and the bogie named as parent. No motor, no brakes: a wagon has nothing to push and the loco does the braking for now. When one bogie node was accidentally given the other's offset the wagon jumped and twisted as soon as it was hauled; fixing the sign fixed it. The real air-brake distributor is the part he had not solved yet. **community**

3 · Parameter links and presets

Besides pin connections, the runtime stores **output-to-input parameter links**. Thomas's:
verified

Output (node.param)	→ Input (node.param)	Meaning
ReverserPosition.OutValue0	ElectricalMotorV2.ReverseFieldPolarity	Reverser flips the motor.
ReverserPosition.OutValue1	DirectBrakeControl.BrakeHoldActive	Reverser in neutral holds the brake.
Power Ramp.Result	TractionBrakingSmoothing.TargetPosition	Shaped throttle becomes the target.
TractionBrakingSmoothing.OutValue0 / 1	ChopperV2.CurrentPosition / DirectBrakeControl.BrakeInputSignal	Positive half = power, negative half = brake (one-lever simple controls).
Axle3.VelocityAtGround_MS	HUD_Accelerometer.InputValue	HUD only.

Presets. Each configuration stores three states: Parked (cold and dark), RunningStopped (ready, brakes on), RunningActive (moving). The game applies one at spawn (`LastAppliedPreset = RunningStoppedPreset`). Thomas's were all unrecorded, so defaults were used. Record yours by setting the graph into each state in the editor and saving; it is what lets a scenario spawn your engine already running.

4 · VHID: a logic half and a click half

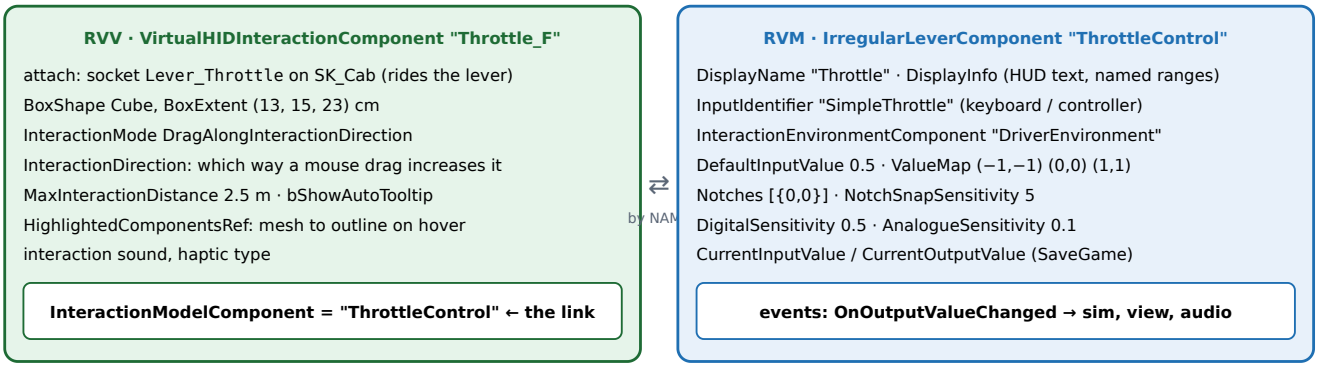


Figure 5.2 · Every control is this pair. The view half is always a VirtualHIDInteractionComponent; the model class changes: IrregularLever for levers, PushButton for buttons (also used for handbrake wheels and door locks), RailVehicleDoor for doors, Coupler / AirHose / JumperCable for connections, Numbering for the running number.
verified

Model-side property (all VHIDs)	Meaning
`InputIdentifier.Identifier`	Keyboard binding: `SimpleThrottle` , `Throttle` , `Reverser` , `TrainBrake` , `LocoBrake` , `Horn` , `Headlights` , `Wipers` , `Sander` , `Bell` . Empty for click-only controls.

Model-side property (all VHIDs)	Meaning
`DisplayInfo`	How the HUD prints it: mode, value source, formatting, named ranges (−1.5..−0.5 “Brake/Reverse”, 0.5..1.5 “Throttle/Forward”).
`bIsSlaveLinked`, `SlaveVHIDComponent`	Rear-cab lever follows the front one.
`ValueMap[]` , `Notches[]` (levers)	Input→output points; detents (a *blocker* needs a second action to pass).
`PushMode` (buttons)	PUSH_AND_HOLD (horn) or TOGGLE (headlights); pressed / released output values.
Door fields	type STAFF/PASSENGER, side, lock state, target state, transition time, delays.

VHID presets on the physics component bundle target values (`HeadlightsOn` : Headlights → 1). The game applies them from the pause menu and during service setup; real locomotives use them for “Cold start”, “Ready to drive”, “Shutdown”.

5 · The three controls a locomotive needs

RVM component	Class	Settings
`ThrottleControl`	IrregularLeverComponent	DisplayName “Throttle”; InputIdentifier `Throttle` (or `SimpleThrottle` with bUsesSimpleControls on the RVD for Thomas's one-lever scheme); environment DriverEnvironment; ValueMap (0,0),(1,1); DefaultInputValue 0; one notch at 0.
`ReverserControl`	IrregularLeverComponent	“Reverser”; `Reverser` ; ValueMap (−1,−1),(0,0),(1,1); notches at −1, 0, 1; DefaultInputValue 0.5; named values Reverse / Neutral / Forward.
`BrakeControl`	IrregularLeverComponent	“Brake”; `TrainBrake` ; ValueMap (0,0),(1,1); DefaultInputValue 1 (brakes on at spawn).
`Horn`	PushButtonComponent	`Horn` ; PUSH_AND_HOLD. Audio only.

On the RVV: one VirtualHIDInteractionComponent per control attached to `SK_RailVehicle` (later `SK_Cab`) at the matching socket, InteractionModelComponent = the RVM component name, Cube hitbox about 10×10×20 cm, DragAlongInteractionDirection, direction +X for forward-is-more. Until the cab exists these are invisible boxes in space; that is fine for testing.

6 · The RVM event graph: glue, not simulation

For each lever, bind `OnOutputValueChanged` and call the set-simulation-parameter node with a struct variable: `Simulation / PowerInput / InputValue` , likewise `ReverserPosition` and `BrakeInput` . Three bindings, three calls, nothing else. Thomas's graph additionally reads `Axle3 / SpeedAtRail_MS` through a threshold delegate (told when speed crosses 0.2 m/s instead of polling), raises

`OnVehicleSpeedChanged` for the view's wheel-blur, and binds the physics component's coupling, door-lock, headlight and preset events. verified

> **Rule of thumb**
> If you find yourself computing physics in Blueprint, that logic belongs in the SimuGraph. The graph copies VHID outputs into SimuGraph inputs, reads a few outputs, and raises events.

7 · What happens to your TSC Lua

TSC Lua	TSW	Notes
`Call("*:GetControlValue", "Regulator", 0)`	Get the VHID's `CurrentOutputValue`, or bind `OnOutputValueChanged`	Event-driven instead of polled.
`Call("*:SetControlValue", "X", 0, v)`	Set the VHID's input value (lever node) or apply a VHID preset	Presets replace most scripted “set several controls” sequences.
`OnControlValueChange(name, index, value)`	One `OnOutputValueChanged` binding per component	No string dispatch.
`Update(dt)` per-frame logic	Simulation component's `OnPreSimulate` delegate, or Tick	Keep it tiny; prefer SimuGraph nodes.
Physics maths (thermal fade, journal friction, adhesion)	`Expression`, `RateOfChange`, `ValueTransition` nodes and parameter links	Integrated at RK4 with the rest of the graph; readable by the ABP, RVA and HUD by name.
Speed-dependent state (dewirement above v, chirp above load)	`RailVehicleParameterChangedDelegate` with a threshold	Fires when a value crosses a target; no polling.
Animation via a control value	A clip in `VirtualHIDSequenceEntries` scrubbed by the VHID	Part 3.
Sounds via control values	RVA binds the same VHID events	Part 6.
`SysCall("ScenarioManager:ShowMessage")`	HUD message / gameplay task system	Different API; keep it for later.
Lead-unit / consist messaging (`SendConsistMessage`)	Hoses and jumper cables: SimuGraph hoses connect graphs across couplers; `ReceiveMessage` on the physics component	The air system crosses vehicles through hose nodes, not messages.

Worked example: the trolley poles

The TSC script (`examples/lua/TrolleyPole.lua`) keeps two poles with states raised / lowered / dewired, raises and lowers them on a control, dewires above a speed and cuts traction while dewired. In TSW:

1. **Controls:** two PushButton VHIDs `PoleFront`, `PoleRear` (TOGGLE) on the RVM, hitboxes on the roof hatch or the cab switch on the RVV.

2. **Animation:** two clips `Pole_F`, `Pole_R` (lowered at frame 0, raised at the end) in `VirtualHIDSequenceEntries`: the pole follows the button's 0..1 output over the button's travel time. Export them from the same keyed objects you made for TSC.
3. **Dewirement:** a `RailVehicleParameterChangedDelegate` on `Axle1.SpeedAtRail_MS` at the dewire speed; in the handler set the button's input back to 0 (pole drops) and set a `ValueConverter` node `PoleContact` to 0.
4. **Traction cut:** a parameter link `PoleContact.OutValue0 → VoltageSource` (or an `Expression` gating `PowerInput`): no contact, no volts. That part of the Lua is now four nodes.
5. **Sounds and sparks:** the RVA binds the pole VHIDs for the clunk; the RVV binds them for a Niagara spark at a `socket_PoleHead` socket.

8 · Simple Physics: the fallback

The physics component carries a complete `SimplePhysics` block (powered flag, max effort, effort-vs-speed curve, max speed, braking effort). It is filled on Thomas even though he has a full graph, and the wiki's setup guide says to fill it for powered vehicles. It is almost certainly what AI-driven and vault-stored vehicles use. Whether a vehicle with no SimuGraph at all is drivable has not been tested; the minimal graph above is cheap, so build it rather than relying on the fallback. Fill the block from your TSC sim blueprint's tractive-effort numbers. **check**